

INSTITUTO TECNOLÓGICO DE TIJUANA

Facultad de Ingeniería
Carrera de Ingeniería en Biomédica

Programación Orientada a Objetos



Arquitectura y Diseño

Proyecto.- Sistema de Ticketera

Alumn@s: Nuñez Luna Vivian Alanis.
Villa Games Legna.
Zetina Sosa Virginia
Profesor: Joel Lee Nation Morales.

Fecha: 8 de mayo de 2026.

1. Introducción.....	3
1.1 Contexto.....	3
1.2 Objetivo.....	3
1.3 Justificación.....	3
2. Desarrollo.....	4
2.1 Diccionario de Clases.....	4
2.1.1 Gestión de Usuarios y seguridad.....	4
2.1.2 Manejo de Tickets.....	6
2.1.3 Control de Eventos y Asientos.....	10
2.1.4 Soporte Tecnico.....	12
2.1.5 Motor de búsqueda.....	14
2.1.6 Sistema de Notificaciones.....	15
2.2 Diseño de Interfaz (Mockups).....	16
2.3 Definición de Excepciones.....	18
Conclusión.....	19
Bibliografía.....	20

1. Introducción

1.1 Contexto

En modismos de estructuración de códigos, un sistema como lo es de ticketera una base sólida en la elaboración de diagramas de Lenguaje Unificado de Modelado o por siglas UML es esencial para la emisión, control y seguimiento de boletos o tickets. Los diagramas UML, el elaborado de diccionario de clases, el diseño de interfaz y la indagación a posibles excepciones permiten visualizar la estructura del sistema, definir responsabilidades y funciones de cada componente así como la antelación a posibles errores de ejecución.

1.2 Objetivo

El reporte se presenta como una investigación y exposición en el diseño de una arquitectura clara y modular en el sistema de ticketera, orientada al análisis y construcción de factores importante para el desarrollo y ejecución del proyecto mostrando la definición de clases y sus relaciones mediante diagramas UML, establecer diccionario de clases con respectivos atributos y métodos; diseño de interfaces intuitivas que faciliten la idea de un esquema interactivo, modesto y cómodo con el usuario así como la implementación de listado de excepciones; todo esto con el fin de garantizar la estabilidad, seguridad y manejabilidad del sistema.

1.3 Justificación

El fin de esto es enfocar el sistema de ticketera como una página confiable, escalable y fácil de mantener. El uso de diagrama UML proporciona una visión estructurada que facilita la comunicación entre desarrolladores y en entendimiento sencillo y conciso de códigos, permitiendo ser información entendible de manera genérica. El diccionario de clases asegura la coherencia, la comprensión y la justificación de cada clase para el logro del sistema así como el diseño de interfaz que nos ayuda a garantizar un modelo sistemático eficiente, accesible y llamativo. Finalmente, se tiene la definición de excepciones para prevención de errores inesperados, manteniendo la integridad de operaciones y reforzar el código con soluciones factibles.

En conjunto, estos elementos forman parte primordial de la inicialización para la constitución de la base metodológica que asegura la calidad del sistema y su adaptación ante técnicos desarrolladores como usuarios.

2. Desarrollo

2.1 Diccionario de Clases

2.1.1 Gestión de Usuarios y seguridad

Clase Base	Rol
Atributos	correo, id_usuario, contraseña, estado_cuenta
Metodos	iniciar_sesion, cerrar_sesion, mostrar_perfil, editar_perfil, recuperar_contra

La clase “Rol” se plantea con el fin de monitorear diversas áreas del sistema, teniendo rangos de accesos y funciones dependiendo del rol.

Clases Derivadas: Cliente, EmpleadoSoporte, Administrador

Clases Derivadas	Cliente	EmpleadoSoporte	Administrador
Atributos	historial_compras tickets metodo_pago	area_soporte atencion_cliente correccion_tickets	nivel_acceso
Métodos	comprar_ticket reservar_ticket cancelar_ticket	responder_cliente cambiar_solicitud generar_reportes_s oporte	crear_evento editar_evento eliminar_evento gestion_usuarios generar_reportes

Se plantea estructuración de clases “Cliente”, “EmpleadoSoporte” y “Administrador”, moldeados y estructurados a fin de atributos y métodos específicos que permitan acatar operaciones según sea el rol. Dividiéndose así en:

- Cliente: tendrá acceso a la compra, reservación y cancelación de tickets propios de eventos seleccionados, asimismo podrá efectuar métodos de pago a elección (tarjeta crédito/débito, PayPal, MasterCard, etc) y acceder a su historial de compras.
- EmpleadoSoporte: responderá a áreas de accesos sistemáticos de página web correspondiente al soporte al cliente, ayudando en la resolución de problemáticas de clientes, respondiendo a estos, dando corrección a tickets, cambiando formatos según solicitud del cliente y generar reportes de dichos soportes, aclarando estados y soluciones.
- Administrador: tendrá diversos niveles de acceso, siendo capaz y responsable de la aclaración y publicación de eventos, pudiendo crear, editar y eliminar estos. Tendrá acceso a la gestión de usuarios, confirmando a través de sus

correos la confirmación de compra de tickets y al registro de soporte de problemáticas presentadas así como a las resolución de estas. Por último, deberá generar reportes de área v de actividades.

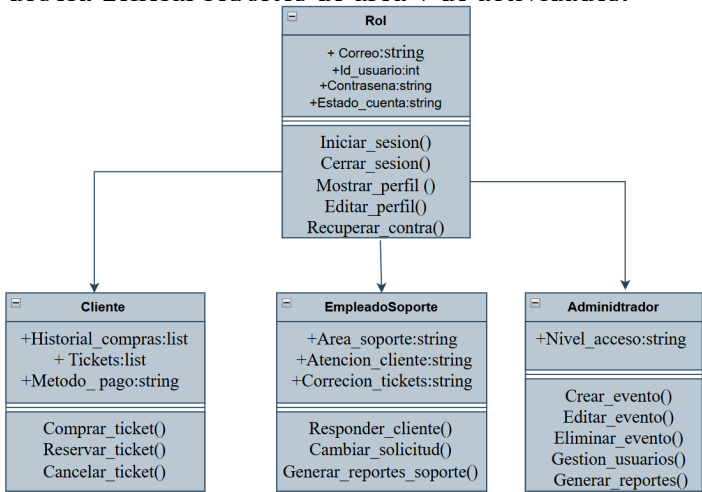


Figura 2.1.1-1. Diagrama de clase “Rol”.

Clase Base: Sesion

Clase Base	Sesion
Atributos	fecha_inicio, hora_inicio, ip_usuario
Métodos	abrir_sesion, cerrar_sesion, validar_sesion

La función principal de esta clase es el acceso de la página a través de la creación e inicio de sesiones, esto con el fin de llevar un control de actividad en la página y que dichos usuarios tengan mayor acceso y accesibilidad a eventos, tickets y programas de la página.

Clases Derivadas: LoginUser, LoginManager

Clase	LoginUser	LoginManager
Atributos	usuario, contraseña	usuario, contraseña usuarios_activos, cuentas_inactivas, intentos_fallidos
Métodos	verificar_correo, error_login, cerrar_sesion_usuario acceso_perfil, notificaciones, menu_principal	verificar_correo, error_login autenticacion_usuario, bloquear_usuario, cerrar_sesion_manager

Las clases derivadas “LoginUser” y “LoginManager” tienen el propósito de permitir el ingreso a la página y acceso a ciertas áreas de acuerdo a la categoría con la que se inicializa la sesión. Siendo así, encontramos:

- LoginUser: a través de esta opción, el sistema clasificará en automático al usuario como cliente, y verifica a través del correo y contraseña si se le permite el acceso sea que los datos dados son coincidentes con la base de datos ya registrados en el sistema o si no; siendo este último en el arrojado de error de ingreso y pidiendo reintentar el proceso hasta lograrlo. De ser aprobado, se le permitirá el acceso a la página y en consecuencia a secciones como lo es el perfil, notificaciones o menú principal.
- LoginManager: el sistema clasificará en automático al usuario como manager, se repetirá el mismo proceso de verificación, hasta lograrlo. De ser aprobado, se le permitirá el acceso a historial de usuarios, observando usuarios activos, cuentas inactivas e intentos fallidos de cuentas determinado la autenticación de

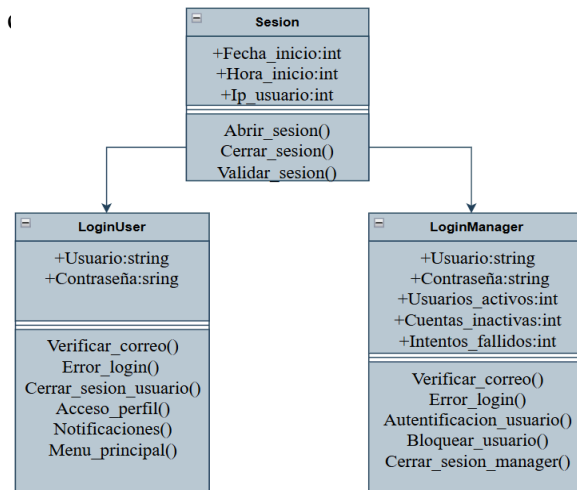


Figura 2.1.1-2. Diagrama UML de clase “Sesion”

2.1.2 Manejo de Tickets

Clase Base: Operacion

Clase Base	Operación
Atributo	fecha_operacion, montos, lugares_evento, estado_operacion
Métodos	registro_operacion, monto_operacion, etapa_operacion

En esta clase se le asigna los atributos de fechas, estados y montos de operación con el fin de otorgar lógica a las funciones de registro, etapa y cantidades manejadas en las operaciones. La clase “Operación” tiene la función de aportar control y manejo de operaciones relacionadas a eventos registrados en la página, sea la supervisión de

fechas estimadas por evento, lugares, montos y costo de operación para la producción y elaboración de dichos eventos así como el seguimiento de etapas de los eventos.

Clases Derivadas: Reservar, Compra, Pago, Factura

Clase Derivada	Reservar	Compra	Pago	Factura
Atributos	tiempo_limite, asiento_reserva do, fecha_y_hora	metodo_pago, folio_compra	tipo_pago, referencia_pa go	folio_factura, fecha_emision, rfc
Métodos	reservar_asiento , cancelar_reserv a	procesar_com pra, confirmar_co mpira	validar_pago, realizar_pago, reembolso	generar_factura, descargar_factur a, enviar_factura

Las clases derivadas se elaboran con la conectividad entre ellas que permiten el seguimiento operacional para la compra y reservación de tickets para un evento específico. Dividiéndose así en:

- Reservar: a través de esta clase se reservara conforme a tiempo, la fecha, hora y asiento de dicho evento, permitiendo la opción de cancelación atemporal sea la circunstancia del cliente.
- Compra: a través de esta clase se permitirá el proceso de compra y la confirmación de cuál sea el método de pago opcional seleccionado brindando información de folio de compra.
- Pago: aquí se dará realización y confirmación de pago, respondiendo a información de pago sea la referencia y tipo de pago aplicado así como la opción de reembolso en todo caso.
- Factura: dada la información requerida para la realización de factura sea RFC, fecha de emisión referencia/folio de pago, se llevará a cabo la generación, descarga y envío de factura vía correo electrónico.

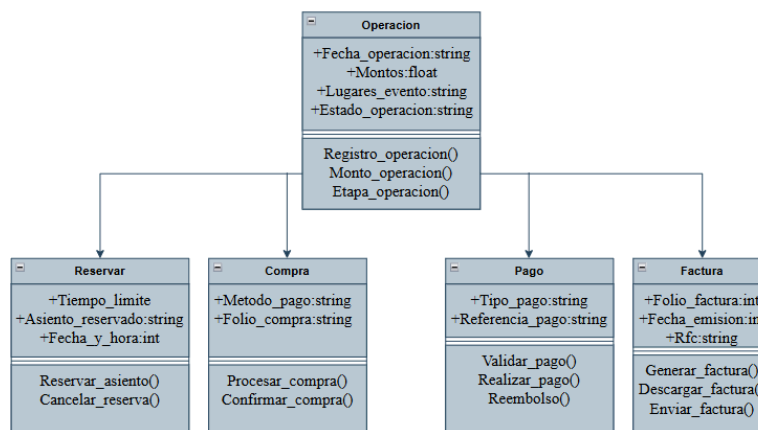


Figura 2.1.2-1. Diagrama UML de clase “Operación”

Clase Base: Categoría

Clase Base	Categoría
Atributo	nombre_categoria, precio, beneficios
Métodos	categoría_ticket (general, VIP, golden), actualizar_precio

Para la clase “Categoría” se pretende, a través de los atributos nombre_categoria, precio y beneficios, brindar las opciones operacionales de categoría de tickets teniendo opción general VIP, golden y actualización de precios.

Clases Derivadas: TicketGeneral, TicketVIP, TicketPreferente

Clase Derivada	TicketGeneral	TicketVip	TicketPreferente
Atributos	(mismos clase base)	(mismos clase base)	(mismos clase base)
Métodos	acceso_general, info_general	acceso_vip, beneficios_vip, info_vip	clasificacion (discapacitado, golden) acceso_area, division_area, info_preferente

Las clases derivadas permiten seleccionar en opciones de tickets, brindando información referente a la categoría dando características de área, accesos y precios. Siendo así, se presenta:

- TicketGeneral: el cual, de obtenerlo, brinda acceso general sea el lugar del evento, delimitada y marcada en mapa interactivo, y brindando información referente al evento como entradas de acceso y salidas.
- TicketVip: brinda acceso a áreas catalogadas VIP, sea por cercanía, disponibilidad o privacidad, otorgando información de beneficios, áreas delimitadas, pases y horarios de entrada.
- TicketPrfenete: sea por ser cliente recurrente, otorgando membresía Golden o sea situaciones específicas y únicas como presentar alguna discapacidad; de igual forma se informa de división de áreas, información genérica de locación y accesos en áreas.

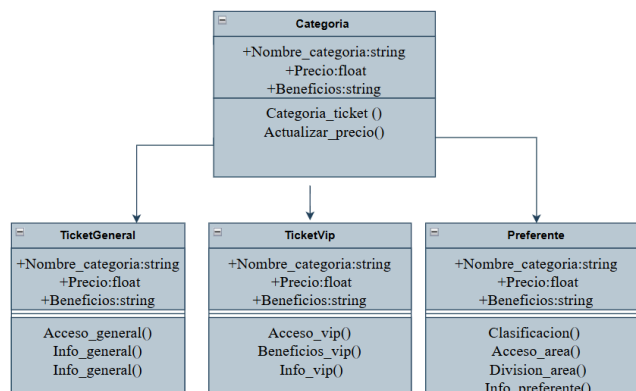


Figura 2.1.2-2. Diagrama UML de clase “Categoría”

Clase: EstadoTicket

Clase Base	EstadoTicket
Atributo	codigo_ticket, no_control
Métodos	estado_tickets (Funciones/def: estado → if/else/elif: disponible, reservado, comprado, cancelado, expirado, usado)

La clase “EstadoTicket” se coloca en adición a la funcionalidad del código. Este estima la etapa o estado del ticket sea marcado como reservado, comprado, cancelado, expirado o usado con el fin de llevar control y manejo a través de código y número de control en el flujo de compra-venta de tickets a en la página.

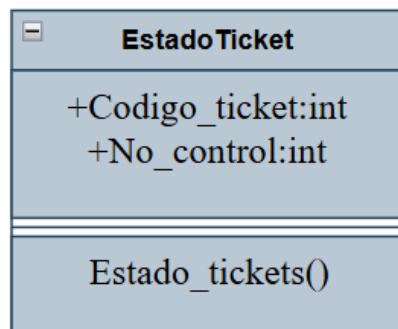


Figura 2.1.2-3. Diagrama UML de clase “EstadoTicket”

Clase Base: ControlTicket

Clase Base	ControlTicket
Atributo	clasificacion, codigo_control_ticket, fecha_generado
Métodos	generar_codigo, validar_activacion_ticket

Esta clase permite el control de tickets a partir de la clasificación , código de control y fecha en la que se generó, permitiendo la generación de código iterativo e irreproducible u alterable, pues el código dinámico permite el cambio constante de este, evitando clonación o reventa de tickets. Al fin de esto se da verificación y validación de ticket.

Clases Derivadas: GeneradorQR, ValidarQR

Clases Derivadas	GeneradorQR	ValidarQR
Atributos	(mismos clase base)	(mismos clase base)

Métodos	crear_qr, actualizar_qr, descarga_qr	escanear_qr, verificacion_qr (validacion o duplicado)
---------	---	---

En base de la validación del ticket, estas clases interactúan con el fin de permitir la generación y validación de código QR para la seguridad y protección tanto del cliente como de la compra.

- **GeneradorQR:** permite la generación de QR dinámico a partir de datos de ticket, dando QR escaneable para la practicidad del cliente y agilización en ingresos de eventos.
- **ValidarQR:** corroborando los datos del ticket y la generación del QR, se da la aprobación del ticket, ayudando a la autenticación de seguridad y control de ventas.

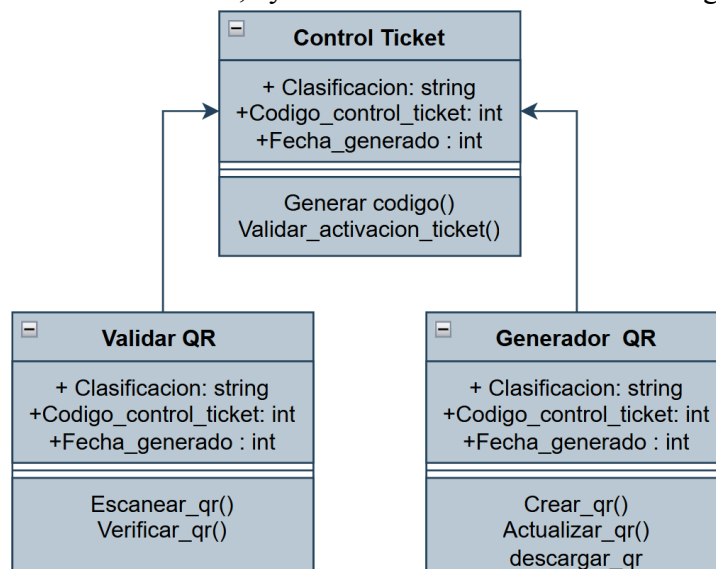


Figura 2.2.2-4. Diagrama UML de clase “ControlTicket”

2.1.3 Control de Eventos y Asientos

Clase Base: Evento.

Clase Base	Evento
Atributo	nombre_evento, fecha, hora, ubicacion, capacidad, estado_evento
Métodos	categoria_evento, limite_capacidad, fecha_limite_evento

El propósito principal de esta clase es el control y administración de eventos mediante atributos (nombre del evento, fecha, hora, ubicación, capacidad y estado del evento) que permitan categorizar, organizar límites de capacidad y fechas de dichos eventos, Permitiendo una dinámica manejable y ágil de sistematización.

Clases Derivadas: Concierto, Convención, FuncionTeatro

Clase Derivada	Concierto	Convención	FuncionTeatro
Atributos	artista, genero_musical	categoria_conven cion, tema	obra, director, categoria_funcion
Métodos	mostrar_artista, reproducir_playlist, info_artista	mostrar_programa , especificacion, datos_recomenda cion	info_funcion

Las siguientes clases derivadas cumplen con la función del manejo categórico a partir de datos específicos que permitan desarrollar nociones e identificaciones conforme a eventos:

- Concierto: permite la exposición de información y datos relevantes como lo es señalar la evolución y formación del artista en cuestión, permitiendo conectar con él mediante la opción de reproducción de álbumes y conociendo un poco más del evento.
- Convención: permite conocer temática y categoría del evento direccionando entre comics, anime y mangas, o videojuegos, permitiendo que el cliente/usuario decida la elección a través del manejo del programa, especificaciones del evento y recomendaciones.
- FuncionTeatro: especializado en obras teatrales, da demostración de la función a partir de información como lo sería nombre de la obra, clasificación, director y género.

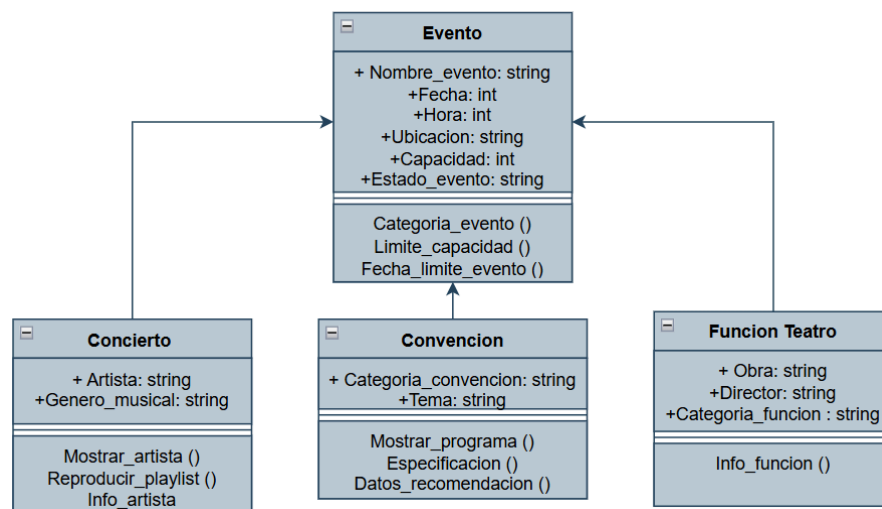


Figura 2.1.3-1. Diagrama UML de clase "Evento".

Clase Base: MapaAsientos

Clase Base	MapaAsientos
Atributo	filas, columnas, asientos_disponibles
Métodos	mostrar_mapa,

	actualizar_asiento, seleccion_asiento, confirmacion
--	---

La clase “MapaAsientos” tiene el objetivo de dar manejo, elección y confirmación de asientos a través de un mapa interactivo dirigido por filas, columnas, y asientos disponibles y que permita señalar la actualización de asientos restantes.

Clases Derivadas: AsientoInteracción, Disponibilidad, Area (general/VIP).

Clase Derivada	AsientoInteraccion	Disponibilidad	Area
Atributos	(mismos clase base)	(mismos clase base)	(mismos clase base)
Métodos	seleccionar_asiento, bloquear_asiento	verificar_disponibilidad, actualizar_disponibilidad	tipo_area, precio_area

Las clases derivadas “AsientoInteraccion”, “Disponibilidad” y “Área” permiten la legibilidad de espacios en los lugares del evento de manera interactiva; proporcionando control de selección a partir del análisis.

- AsientoInteraccion: permite seleccionar asientos y remarcar la zona para evitar la sobre elección o confusión de asientos.
- Disponibilidad: marca la disponibilidad de asientos, siendo constantemente verificado y actualizado para comodidad, comprensión y facilidad del cliente.
- Área: da información precisa acerca del nombre y tipo de áreas de asientos manejadas en el evento y los precios conforme a cercanía, legibilidad, disponibilidad, etc.

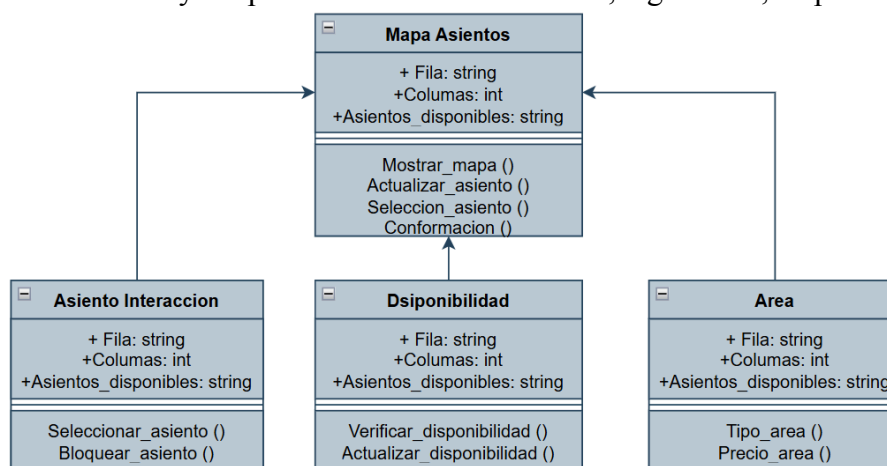


Figura 2.1.3-1. Diagrama UML de clase “MapaAsientos”.

2.1.4 Soporte Tecnico

Clase Base: SoporteTecnico

Clase Base	SoporteTecnico
------------	----------------

Atributo	mensaje, prioridad, estado_solicitud, fecha_solicitud
Métodos	crear_solicitud, modificar_solicitud, mostrar_estado,

Esta clase permite que encargados en soporte técnico tengan accesibilidad y gestión de solicitudes, sean dadas por dudas y problemáticas del cliente así como arrojadas por el mismo sistema, autorizando la creación, modificación y observar la etapa de la solicitud.

Clases Derivadas: Solicitud(categoría de ayuda)

Clases Derivadas	Solicitud	Chat
Atributos	categoria_ayuda, descripcion	mensajes, usuario_emisor
Métodos	enviar_solicitud, editar_solicitud	enviar_mensaje, recibir_mensaje, finalizar_chat

Estas clases derivadas, mediante su correlación, gestionan operaciones que derivan una de la otra, siendo que mediante la solicitud se transmite el mensaje mediante un chat interactivo.

- Solicitud: permite categorizar el tipo de ayuda en base a la descripción dada, enviando y editando así dicha solicitud, para fines administrativos y organizativos.
- Chat: permite la emisión, recepción y envío de mensajes de usuarios que sean con fines recomendativos o reseñas que nos ayuden a entender a nuestros usuarios y mejorar nuestro sistema.

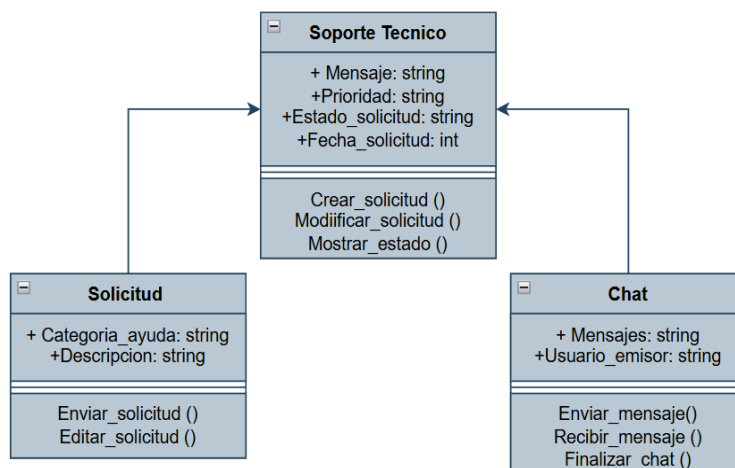


Figura 2.1.4-1. Diagrama UML de la clase “SoporteTecnico”.

2.1.5 Motor de búsqueda

Clase Base: Filtro

Clase Base	Filtro
Atributo	tipo_filtro
Métodos	aplicar_filtro, limpiar_filtro

La clase “Filtro” tiene el propósito de filtrar opciones de búsqueda según sea el tipo o categoría, derivando de eventos como conciertos, convenciones y funciones en componentes de género, artistas, temáticas, precio, lugar, etc.

Clases Derivadas: FiltroFecha, FiltroCategoría, FiltroPrecio, FiltroUbicación

Clase Derivada	FiltroFecha	FiltroCategoría	FiltroPrecio	FiltroUbicación
Atributos	(mismos de clase base)	(mismos de clase base)	(mismos de clase base)	(mismos de clase base)
Métodos	filtrar_por_fecha	filtrar_por_categoria	filtrar_porPrecio	filtrar_porUbicación

Las clases derivadas cumple el propósito de filtrar la búsqueda según la opción de clasificación encontrando:

- FiltroFecha: permite enfocar la búsqueda según fechas de eventos.
- FiltroCategoría: permite enfocar la búsqueda según la categoría del evento, sea conciertos, convenciones o funciones teatrales.
- FiltroPrecio: permite enfocar la búsqueda según el precio del evento, arrojando eventos previamente asistidos como nuevos.
- FiltroUbicación: permite enfocar la búsqueda según lugar y ubicación del evento.

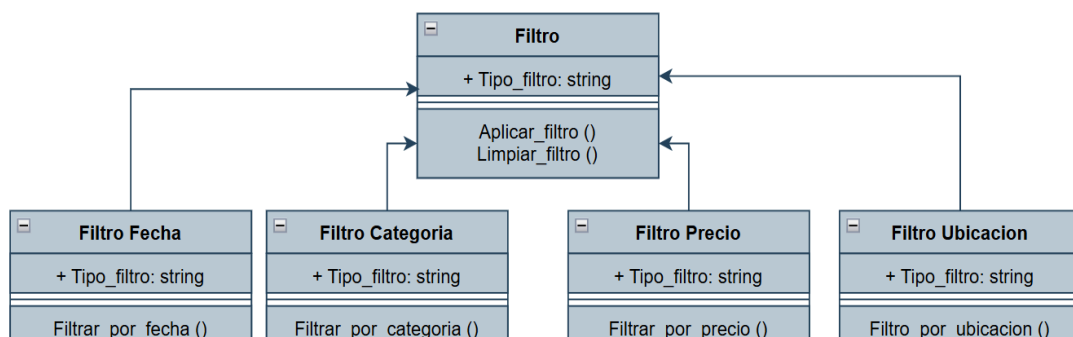


Figura 2.1.5-1. Diagrama UML de la clase “Filtro”.

Clase: Buscador

Clase	Buscador
Atributo	palabra_clave, resultados
Métodos	buscar_evento, motrar_resultados, ordenar_resultados

Esta clase permite la búsqueda de eventos, a través de palabras claves para la muestra ordenada de resultados, dando posibilidad de selección y búsqueda completa de entre diversas opciones y datos.

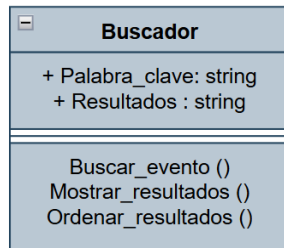


Figura 2.1.5-2. Diagrama UML de la clase “Buscador”.

2.1.6 Sistema de Notificaciones

Clase Base: Notificación

Clase Base	Notificacion
Atributo	mensaje, fecha_envio, estado_envio, destinatario
Métodos	enviar_notificacion, mostrar_notificacion

La función principal de esta clase es la supervisión y aviso de noticias referente a problemáticas, cambios y anexos en los eventos, sea que se haya cancelado un evento, cambiado fechas por un imprevisto, nuevas promociones, o eventos próximos, así como si se presenta errores en algún proceso de reservación, pagos u obtención de tickets. Todo será notificado al cliente para su seguridad, comodidad y confianza.

Clases Derivadas: CorreoNoti, SMSNoti, NotiManager

Clase Derivada	CorreoNoti	SMSNoti	NotiManager
Atributos	correo_destinat ario, asunto	numero_telefonico	lista_notificaciones
Métodos	responder_corre o, marcar_visto	responder_sms, enviar_respuesta	programar_notoficacion, cancelar_notificaicion, mostrar_historial_noti

Las siguientes clases derivadas, formulan con el fin de brindar información base a ciertas opciones operacionales que permitan transmitir y dar noción de cambios u

imprevistos, así como notificaciones por diversos medios, esto proporcionado por personal autorizado.

- CorreoNoti: pretende ofrecer información por medio de correo electrónico permitiendo la interacción por medio de la comunicación.
- SMSNoti: siendo que el usuario brinde número de teléfono en su perfil, igualmente se le podrá proporcionar la información por este medio, si así lo prefiere.
- NotiManager: aquí se otorga, sea por lista de notificaciones, el manejo, administración y gestión de notificaciones, logrando hacer programación, cancelación y muestra de historial.

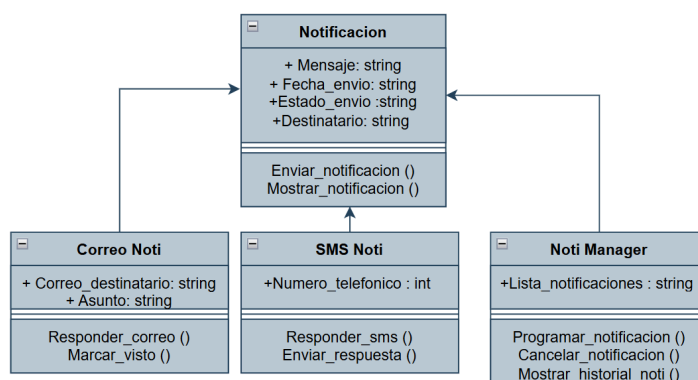


Figura 2.1.6-1. Diagrama UML de clase “Notificacion”.

2.2 Diseño de Interfaz (Mockups)

Las imágenes muestran el diseño de la interfaz del sistema de ticketera, donde se aprecian las distintas opciones disponibles para el usuario. En la primera se observa el menú principal, que organiza las funciones por categorías como Inicio, Iniciar sesión, Evento y Tickets. Cada sección despliega subopciones que permiten acceder a distintos tipos de usuarios —cliente, empleado y administrador—, así como a la gestión de eventos de concierto, teatro o convención, y a los tipos de boletos disponibles: general, VIP y preferente.

La segunda imagen complementa la estructura mostrando las áreas operativas del sistema. Aquí se incluyen módulos como Asientos, donde se controla la disponibilidad y ubicación; Operación, que abarca procesos de reserva, compra, pago y facturación; QR, destinado a la generación y validación de códigos; y Soporte técnico, que ofrece canales de atención mediante solicitud o chat.

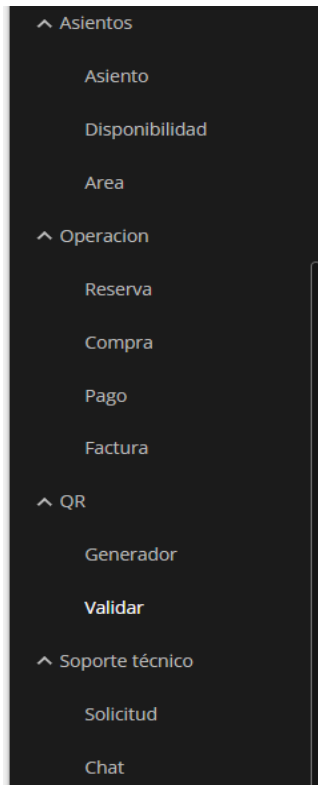


Figura 2.2.-1 Diseño de interfaz

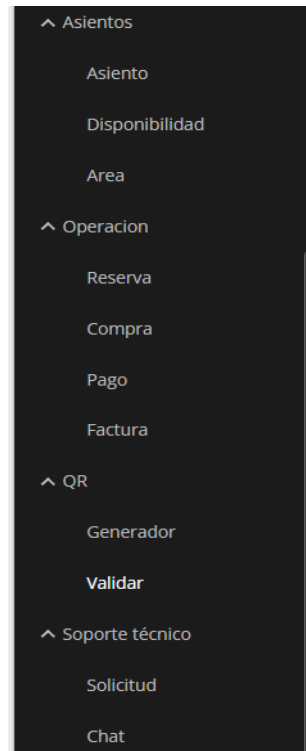


Figura 2.2-2 Diseño de interfaz

Imágenes interactivas que direccionan, conforme a la sección y concepto; a áreas de categorías específicas; permitiendo la demostración informativa del sector y dando desglose del programa en cuestión como lo sería una promoción nueva, descripción del programa de un concierto, un evento nuevo o el estreno de una nueva obra teatral.

No solo compras una entrada, aseguras la experiencia.



CANTANTES

Se estrenará la película de su gira "Hit Me Hard and Soft: The Tour (En vivo en 3D)" en cines el 8 de mayo de 2026. La cinta fue grabada en 3D inmersivo durante su gira mundial, ofreciendo una experiencia visual de sus conciertos.



VIDEOJUEGOS

Colonia, Alemania. Del 26 al 30 de agosto de 2026, consolidándose nuevamente como el mayor evento mundial de videojuegos. La feria ofrecerá cinco días de lanzamientos, tecnología y cultura gamer, inaugurada por el espectáculo *Gamescom: Opening Night Live*



TEATRO

Nueva York, Times Square. Es uno de los musicales más exitosos y aclamados de Broadway, destacando por sus impresionantes marionetas, máscaras y vestuarios diseñados por Julie Taymor. La obra, actualmene en cartelera, ofrece un espectáculo familiar de 2.5 horas con un intermedio.

Figura 2.2-3. Diseño de Interfaz. Imágenes interactivas.

2.3 Definición de Excepciones

2.3.1 Error de inicio de sesión

Problema:

El usuario ingresa un correo o contraseña incorrectos al intentar acceder al sistema.

Solución:

Se implementará una validación de datos que compare la información ingresada con la base de datos. En caso de error, se mostrará un mensaje indicando que las credenciales son inválidas

2.3.2 Compra de boletos sin disponibilidad

Problema:

El usuario intenta comprar o reservar boletos cuando ya no hay asientos disponibles en la zona seleccionada.

Solución:

Antes de confirmar la compra, el sistema validará la disponibilidad en tiempo real. Si no hay espacios, se mostrará un mensaje indicando que la zona está agotada.

2.3.3 Intento de acceso con ticket inválido o duplicado (QR)

Problema:

Se intenta validar un ticket falso, duplicado o ya utilizado en el acceso al evento

Solución:

El sistema de escaneo verificará el estado del ticket. Si ya fue usado o no existe, se rechazará el acceso y se notificará como inválido.

2.3.4 Búsqueda sin resultados

Problema:

El usuario realiza una búsqueda de eventos sin coincidencias.

Solución:

Se mostrará un mensaje indicando que no hay resultados y se sugerirán opciones similares o filtros alternativos.

Conclusión

El desarrollo del sistema de ticketera permitió analizar la importancia de aplicar una estructura organizada y modular dentro del diseño de software. A través de la elaboración de diagramas UML, diccionario de clases, diseño de interfaces y definición de excepciones, se logró establecer una base sólida para la construcción de un sistema funcional, seguro y eficiente. Cada una de las clases propuestas cumple un papel específico dentro de la arquitectura, permitiendo la correcta administración de usuarios y manejo de tickets.

Asimismo, la implementación de relaciones entre clases base y derivadas facilita el poder reutilizar el código, permitiendo que futuras mejoras puedan integrarse de manera más sencilla. El uso de métodos y atributos bien definidos contribuye a una mejor comprensión del funcionamiento interno del proyecto, favoreciendo tanto a desarrolladores como a usuarios finales.

Por otra parte, el diseño de interfaces y el manejo de excepciones representan elementos fundamentales para garantizar una experiencia de usuario más intuitiva, estable y confiable. La prevención de errores comunes, como accesos inválidos, boletos duplicados o búsquedas sin resultados, fortalece la seguridad del sistema y mejora la interacción con la plataforma. En conclusión, este proyecto demuestra cómo la planificación y modelado previo mediante herramientas de análisis y diseño orientado a objetos permiten desarrollar sistemas más organizados, comprensibles y adaptables, sentando las bases necesarias para la futura implementación de una ticketera moderna y eficiente.

Bibliografía

- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The unified modeling language user guide* (2nd ed.). Addison-Wesley.
- Django Software Foundation. (2026). *Django documentation*. <https://docs.djangoproject.com/>
- FastAPI. (2026). *FastAPI documentation*. <https://fastapi.tiangolo.com/>
- Flask Documentation. (2026). *Flask documentation*. <https://flask.palletsprojects.com/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3rd ed.). Prentice Hall.
- Object Management Group. (2026). *Unified Modeling Language (UML)*. <https://www.omg.org/spec/UML/>
- Oracle. (2026). *Object-oriented programming concepts*. <https://docs.oracle.com/javase/tutorial/java/concepts/>
- Python Software Foundation. (2026). *Python documentation*. <https://docs.python.org/3/>
- Reflex. (2026). *Reflex documentation*. <https://reflex.dev/docs/getting-started/introduction/>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Ticketmaster. (2026). *Ticketmaster official website*. <https://www.ticketmaster.com/>
- W3Schools. (2026). *Python classes and objects*. https://www.w3schools.com/python/python_classes.asp